

joint trajectory or tcp trajectory will be smoother (no stoppage while executing motion on robot) on executing motion on robotic arm?

Both joint and TCP trajectories can execute smoothly without stopping. The main factor is how the trajectory is generated and executed—not whether it is represented in joint space or TCP space.

Aspect	Joint-space trajectory	TCP / Cartesian trajectory
What you specify	Joint angles over time, $q(t)$	Tool position and orientation over time
Main advantage	Direct control of joint motion and its smoothness	Control of the tool's path relative to the workpiece
Main limitation	Smooth joints do not guarantee the required TCP path or constant TCP speed	A smooth TCP path can demand rapid joint motion near singularities
Can it run without intermediate stops?	Yes	Yes

For example, UR supports blending between motion segments so the robot can pass through a waypoint region without stopping. This applies to both joint and Cartesian motions, subject to controller restrictions.

What actually prevents stop–start motion?

1. **Continuous execution:** Send a complete trajectory, or maintain a buffered stream. Sending a point, waiting for completion, then sending the next often causes stops.
2. **Suitable waypoint velocities:** Do not require zero velocity at every intermediate waypoint.
3. **Smooth transitions:** Use continuous curves or blending within the permitted path tolerance.
4. **Feasible joint motion:** Respect joint velocity, acceleration and jerk limits; avoid abrupt IK configuration changes and singularities.

An exact sharp corner cannot generally be traversed at nonzero speed with bounded acceleration. You must either slow to a stop or round the corner. Blending changes the path near the waypoint, so its tolerance matters for inspection.

For surface inspection, I would use this approach:

TCP tool path → continuous IK / constrained joint path → time parameterization and smoothing → continuous execution.

This lets you preserve the required surface coverage, tool orientation and stand-off distance while producing feasible joint motion. After interpolation or smoothing, verify that the TCP still follows the inspection path within tolerance.

So, joint trajectories are not automatically smoother. For inspection, define the required motion in TCP space and ensure the resulting joint trajectory executes continuously.

Which robot and execution interface are you using—FANUC, UR, ROS 2 control, or Isaac Sim?

for motion generation we use rmpflow and execution on robot we used Fanuc RMI

With RMPflow + FANUC RMI, switching between joint and TCP points alone will not eliminate stoppages. I would first check the RMI blending, buffering and point spacing.

There are two motion-generation stages in your setup:

- **RMPflow** generates joint position and velocity targets from the robot state and a TCP target. It is a reactive motion policy.
- **FANUC RMI** accepts motion instructions; the FANUC controller generates the motion between their endpoints. Consequently, sending RMPflow positions through RMI does **not automatically reproduce RMPflow's original velocity profile or timing.**

Joint coordinates and joint motion are different things. RMI supports:

RMI instruction	Endpoint supplied as	Motion between endpoints
<code>FRC_JointMotionJRep</code>	Joint angles	Joint-interpolated motion
<code>FRC_LinearMotionJRep</code>	Joint angles	Linear TCP motion
<code>FRC_LinearMotion</code>	Cartesian pose	Linear TCP motion

Thus, you can supply joint angles while requesting linear TCP motion.

For your inspection path, my recommendation is:

- Use **linear TCP motion with controlled blending** where tool-path accuracy and scan speed matter.
- Use **joint motion** for transfers where the TCP's intermediate path is flexible.
- Validate the executed path: either interpolation method can change the path between RMPflow samples.

For the actual stop—start issue, check these first:

Check

Why it matters

Intermediate points use **CNT**, **FINE** requests a stop; **CNT** permits blending rather than **FINE**

Several instructions remain queued

Sending one instruction and waiting for completion before sending another prevents useful look-ahead.

Points are not excessively close together

Very short segments can force controller slowdowns.

FANUC manual **B-84184EN/01** specifies an eight-instruction buffer and a **40 ms minimum motion duration**. These are version-specific limits to verify against your controller.

For example, **if you turn every 10 ms RMPflow sample into a separate RMI move**, that older RMI limit would prevent matching the intended timing. You would need suitable resampling while preserving path accuracy. Increasing **CNT** alone is insufficient, and excessive blending can cut inspection corners.

Also check your RMPflow target logic: **waiting for the robot to settle at every TCP target can introduce deceleration before the data even reaches RMI**. A smoothly advancing target is more appropriate for continuous traversal, with tracking error monitored.

Can you share one RMI motion JSON packet and the loop that sends it? That will reveal the instruction type, termination setting and whether you are keeping the queue filled.

The difference is what triggers the next target: the robot reaching a point, or a planned clock. My earlier wording was too abstract.

Imagine your JSON contains three points along a straight line:

A = 0 mm, B = 10 mm, C = 20 mm.

What she may be doing now

1. Set RMPflow's target to **B**.
2. Keep B as the target while the robot approaches it.
3. Wait until the robot reaches B—possibly also waiting until it stops.
4. Set the target to **C**.

For that whole first movement, RMPflow only knows: **“Go to B.”** It does not know that your application intends to continue to C. It therefore tends to slow down as it approaches B.

Then the target changes to C, and the robot accelerates again.

What I suggested instead

Give RMPflow a target that moves gradually along the line:

- First, target **1 mm**.
- A short time later, target **2 mm**.
- Then **3 mm, 4 mm**, and so on.
- After **10 mm**, continue to **11 mm, 12 mm...**
- Stop advancing the target at the final destination, **20 mm**.

We update the target while the robot is still moving. We do not wait for it to reach and stop at each target.

Think of a robot following a moving marker. The marker keeps travelling along the required path, and RMPflow makes the robot follow it.

	Her possible current approach	Proposed test
Targets	Original JSON points: B, then C	Intermediate positions along A–B–C
When target changes	After the robot reaches the current point	At each scheduled update
At B	Wait for arrival, possibly settling	Continue moving the target towards C
Main thing to check	Repeated deceleration near points	Whether the robot keeps up accurately

Where do the intermediate positions come from?

The code calculates them between the JSON points. For this example, between **0 mm and 10 mm**, the positions **1, 2, 3... mm** are simply points on the same line. That calculation is called *interpolation*.

The spacing and update interval determine the target's speed:

1 mm every 20 milliseconds = 50 mm/second.

This is an example, not a speed recommendation. The robot's actual speed may differ because it takes time to follow the target.

What you can explain to her

“You may currently be treating every JSON waypoint as a separate destination. Please check whether the code waits for the robot to arrive or stop before selecting the next waypoint. If so, let's test continuously advancing the target along one straight section instead. Calculate

intermediate positions between the JSON points and update the target regularly while the robot is moving. Then check whether the robot follows accurately without the repeated pauses.”

First confirm her current logic. If she already advances targets before the robot settles, the pauses may have another cause. And this moving-target approach needs tracking-error checks; it does not automatically guarantee accurate inspection motion.

is it something like interpolation we need to do between the existing points?

Yes—interpolation between the existing waypoints is part of it. But the crucial change is to update the target continuously, without waiting for the robot to settle at each point.

For example:

- **Existing JSON points:** 0 mm → 10 mm → 20 mm.
- **Interpolated targets:** 0 → 1 → 2 → 3 ... → 20 mm.
- **Target update:** give RMPflow the next interpolated target at a defined time interval, while the robot is moving.

Approach	Expected effect
Original points + wait at each point	Can cause repeated slowing or stopping
Interpolated points + wait at each point	Can still cause repeated slowing—just at more points
Interpolated points + continuous timed updates	Provides a moving reference for RMPflow to follow

So you can tell her:

“Let’s interpolate the TCP path between the JSON waypoints and advance the RMPflow target at each simulation step, according to a chosen speed, instead of waiting for convergence at every waypoint.”

Two distinctions matter:

- **Interpolation does not automatically smooth sharp corners.** Those need suitable rounding within the allowed path tolerance, or a slowdown.
- **This is for feeding RMPflow.** It does not mean sending every interpolated point as a separate FANUC RMI instruction.

Start with a straight segment and fixed tool orientation, then verify tracking error and TCP speed.

now can you list down all suggestions (like what she has to do) and why are we doing it?

Ask her to investigate in this order: current waypoint logic → RMPflow output → FANUC RMI execution. Change one thing at a time so you can identify what actually removes the pauses.

A. Check how the JSON waypoints are given to RMPflow

What she has to do	Why we are doing it
1. Inspect the waypoint-switching condition. Check whether the next waypoint is selected after position convergence, near-zero velocity, or a fixed delay.	Waiting for the robot to settle can create a slowdown or stop at every waypoint.
2. Choose one straight section with fixed tool orientation as a test case. Record the current behavior before changing anything.	This isolates waypoint handling from complications such as corners, orientation changes and singularities.
3. If the current logic waits at each point, implement interpolation between the JSON waypoints. Calculate intermediate TCP positions along the existing path.	This provides intermediate targets that can advance gradually instead of jumping between distant destinations.
4. Advance the interpolated target at each simulation step, according to a chosen reference speed. Do not wait for convergence at intermediate points.	Interpolation alone will not remove pauses. The target must continue advancing while the robot moves.
5. Ramp the reference speed up at the start and down at the end.	An immediate jump from zero to scan speed creates an abrupt reference change.
6. Measure actual TCP tracking error. Compare the simulated TCP with the moving reference and the required inspection path.	The robot may lag behind or cut corners. Continuous motion is useful only if inspection accuracy remains acceptable.

For step 4, the relationship is:

Target advance per update = reference speed × simulation time step

For example, **50 mm/s × 0.01 s = 0.5 mm per update**. This is illustrative, not a recommended operating speed.

B. Check timing and smoothness of the generated motion

What she has to do

Why we are doing it

7. Verify the time step used to compute RMPflow actions matches the simulation physics step. Log timestamps with generated samples.

Joint positions without consistent timing do not define a reliable execution-speed profile.

8. Plot reference TCP speed and simulated TCP speed against time. Mark the original waypoint locations.

This shows whether speed drops originate in the reference or in the robot's response, and whether they occur at waypoint boundaries.

9. Plot generated joint positions, velocities and accelerations. Check continuity and configured limits; inspect jerk if acceleration changes abruptly.

A continuous TCP reference can still produce abrupt or infeasible joint motion.

10. At remaining problem locations, inspect sharp corners, orientation changes, joint limits and singularities.

These can require slowing down even when waypoint handling is correct.

RMPflow's internal integration substep is a numerical setting; it is **not a scan-speed setting or a guarantee of correct trajectory timing**.

For a curved inspection path, any rounding or smoothing must stay within the permitted TCP position, orientation and stand-off tolerances.

C. Check how the generated motion is converted into FANUC RMI instructions

What she has to do

Why we are doing it

11. Identify the exact RMI motion instruction used. Check whether it requests joint or linear TCP interpolation.

Sending joint angles does not necessarily mean joint-interpolated motion. The instruction determines the motion between endpoints.

12. Log `TermType` and `TermValue`. Check intermediate `FINE` instructions and use appropriate `CNT` blending where continuous motion is intended.

Intermediate termination settings can request stops. Excessive blending can also introduce unacceptable path deviation.

13. Inspect the instruction queue. Log sent/completed sequence IDs and timestamps; keep upcoming instructions buffered within the supported capacity.

Waiting for each motion to finish before sending the next can deprive the controller of the upcoming motion needed for continuous execution.

14. Check segment spacing and expected duration. Avoid converting every high-frequency RMPflow sample directly into a tiny RMI move.

Very short instructions can cause controller-imposed slowdowns. Resampling must preserve path accuracy.

15. Document how generated timing is translated into RMI speed or duration fields.

Sending positions alone does not reproduce the original RMPflow velocity profile or timing.

16. Compare actual FANUC TCP speed and path accuracy with the simulation.

Smooth simulation does not establish that the physical robot executes the same motion smoothly or accurately.

FANUC's older RMI manual, B-84184EN/01, documents an eight-instruction buffer and a 40 ms minimum move duration. She should confirm the applicable limits for your installed controller version.

What to ask her to bring back after the first trial

- The current waypoint-switching condition.
- One example RMI motion packet and a description of the buffering logic.
- Reference, simulated and actual TCP speed plots for the same short path.
- Tracking/path error and the locations of unintended stops.
- Which single change was tested and its effect.

Start with steps 1–2 and 11–13. They may reveal an explicit settling condition, intermediate stop instruction or buffering issue before any interpolation changes are needed.